

SEC-PRJ-7F_25: Triple Factor Authentication for Social Media

Beishenaliev Ermek 563075

1. Introduction

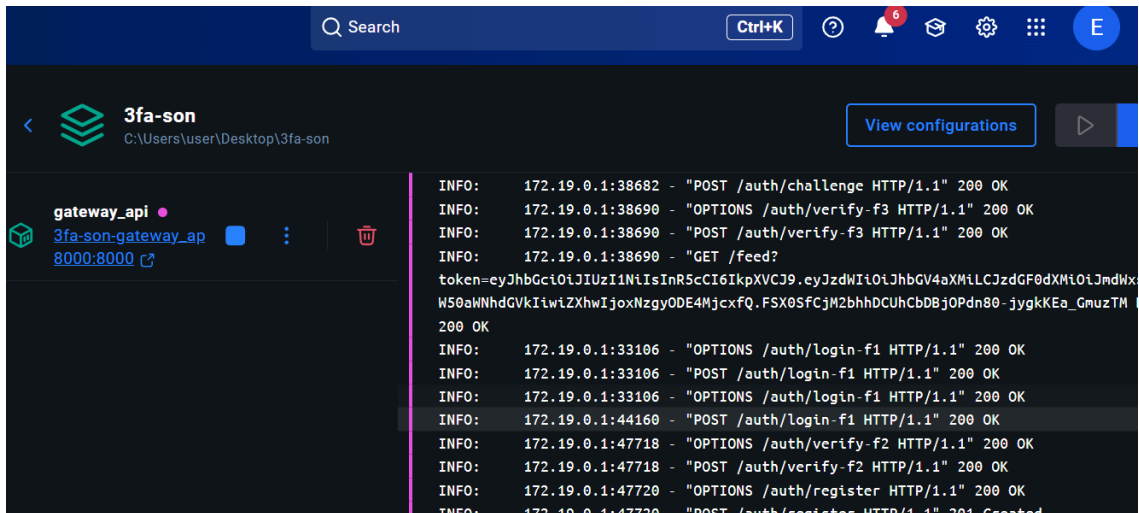
Modern social media platforms face sophisticated cyber threats, rendering traditional single-factor authentication obsolete. As remote credential-stuffing and phishing attacks become increasingly automated, securing user data requires a paradigm shift in identity verification. This project demonstrates a highly secure, enterprise-grade Triple Factor Authentication (3FA) Gateway.

The system enforces strict Zero-Trust principles by requiring proof of Knowledge, Possession, and Inherence (Trusted Device) before granting access to a protected JSON Web Token (JWT) session. By implementing these three distinct pillars of authentication, the system ensures that the compromise of any single factor—or even two factors—does not result in a system breach.

2. Architecture Overview

The architecture is designed to be lightweight, stateless, and mathematically secure, utilizing standard cryptographic protocols rather than relying on vulnerable third-party APIs. The decoupling of the client and server ensures maximum flexibility and security isolation.

- **Frontend Client:** A single-page application (HTML/CSS/JS) utilizing the native browser Web Crypto API for local key generation.
- **Backend API Gateway:** A Python-based FastAPI microservice handling asynchronous cryptographic verification.
- **State & Storage:** An SQLite database ([3fa.db](#)) storing user credentials, public keys, and one-time cryptographic challenges.
- **Session Management:** Stateless JWT (JSON Web Tokens) signed with the HS256 algorithm.
-



3. Cryptographic Primitives & Theoretical Foundation

To understand the security guarantees of this gateway, it is necessary to define the underlying cryptographic algorithms utilized across the three authentication factors.

Argon2ID Key Derivation

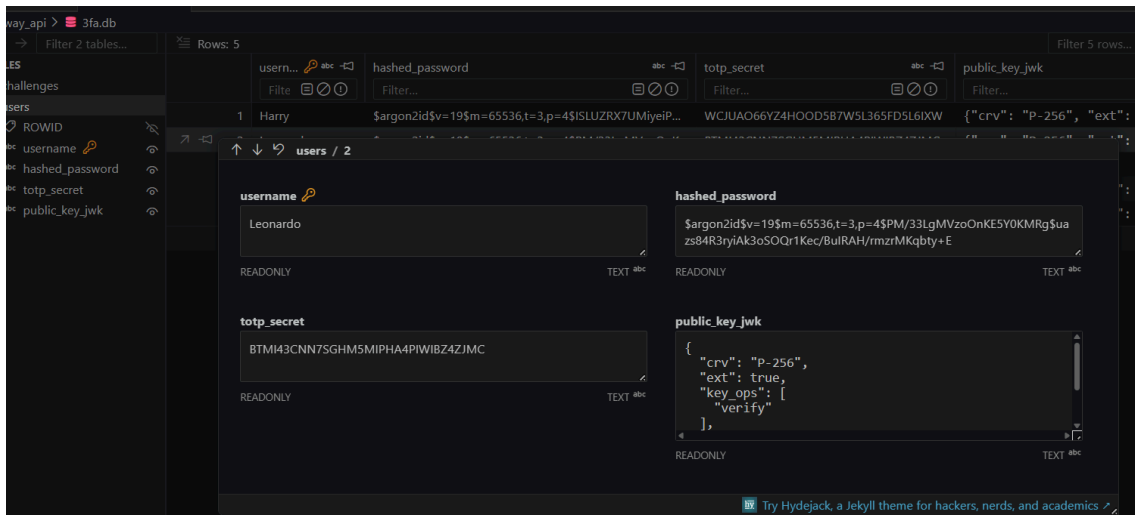
Argon2ID is utilized for password hashing. Unlike legacy algorithms such as MD5 or SHA-256, Argon2ID is a hybrid memory-hard function. It is specifically designed to resist parallelized brute-force attacks from GPUs and ASICs. By configuring the memory cost, time cost, and parallelism parameters, the server intentionally slows down the hashing process to thwart offline dictionary attacks.

Time-Based One-Time Password (TOTP)

The possession factor relies on the RFC 6238 standard. The backend utilizes the HMAC-SHA1 algorithm to verify a 6-digit code against a moving 30-second time window. The algorithm fundamentally relies on a shared secret key K and a moving time counter C .

Elliptic Curve Digital Signature Algorithm (ECDSA)

For device inheritance, the system employs ECDSA over the P-256 curve. This asymmetric cryptographic algorithm allows the client to generate a signature using a securely stored Private Key. The server can verify this signature using the corresponding Public Key without ever possessing the secret.



4. The Triple-Factor Authentication Protocol

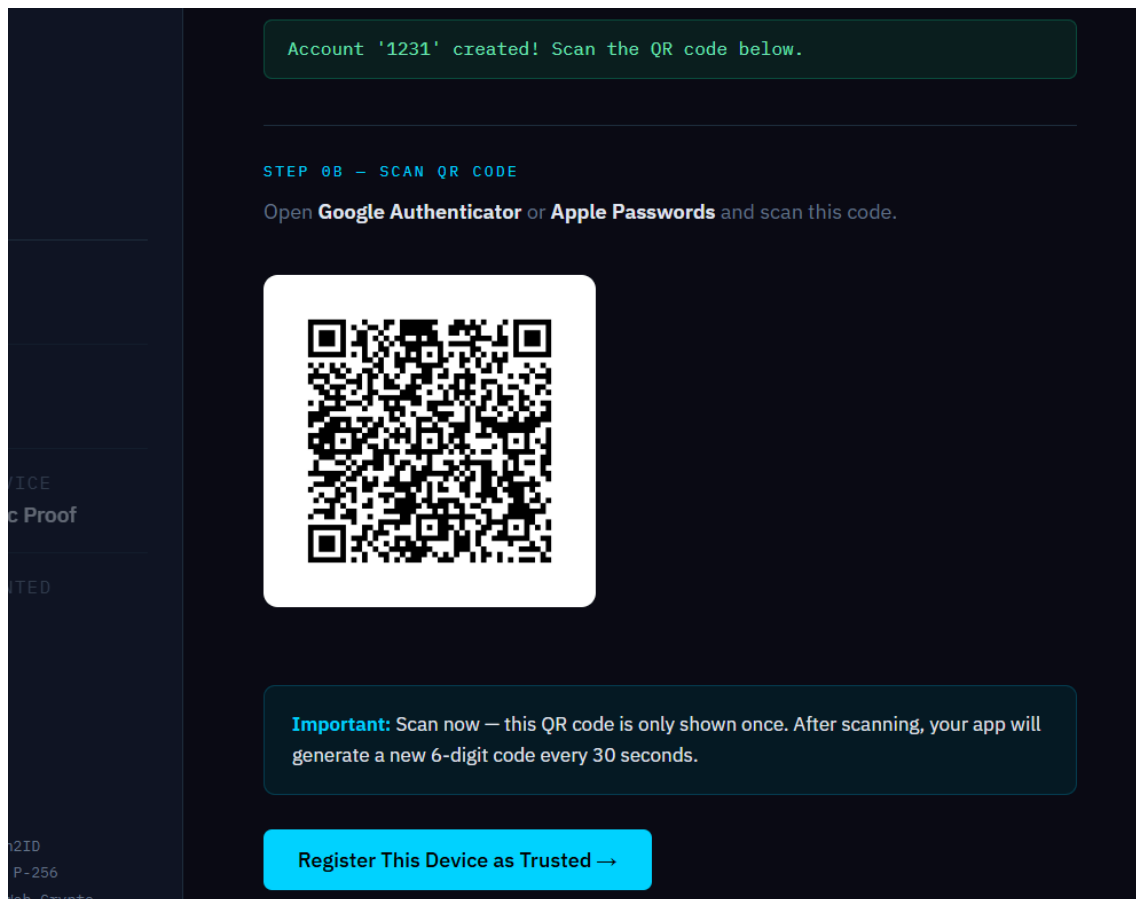
The gateway strictly implements three independent authentication factors to prevent unauthorized access. The protocol operates as a state machine, upgrading the user's access privileges only as each cryptographic proof is verified.

Factor 1: Knowledge (The Password)

Upon registration, the user's password is not stored in plaintext. It is hashed via Argon2ID and saved to the SQLite database. During the login phase, the user submits their plaintext password, which the FastAPI backend hashes and compares against the database record. Upon successful verification, a temporary JWT is issued with a `pending_f2` status.

Factor 2: Possession (The TOTP Code)

Following password verification, the system demands proof of physical device possession. During initial registration, the user pairs their account with an authenticator application (e.g., Google Authenticator) via a generated QR code.

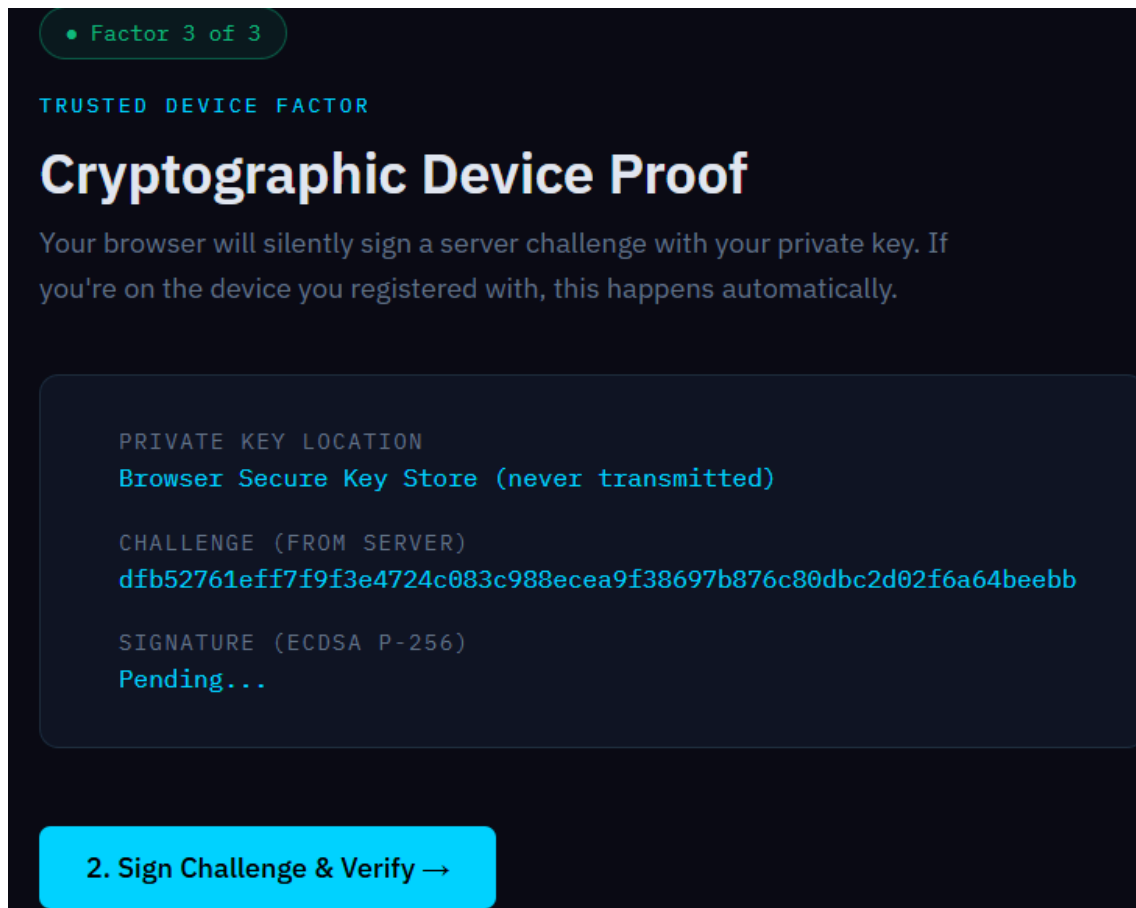


During login, the user must provide the current 6-digit code. This ensures that even if a password is stolen, the attacker cannot proceed without physical possession of the user's mobile device. Successful validation upgrades the JWT state to `pending_f3`.

Factor 3: Inherence & Trusted Context (Cryptographic Proof)

The final factor ensures the login request originates from a registered "Trusted Device," neutralizing remote credential-stuffing attacks.

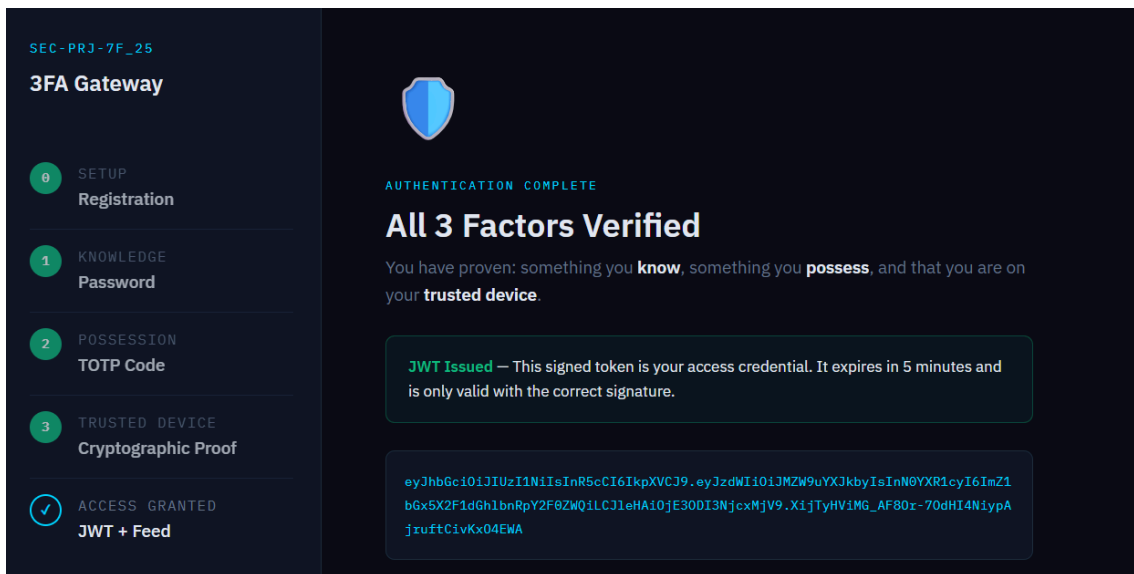
1. **Key Generation:** During registration, the browser's Web Crypto API generates an ECDSA P-256 Keypair. The Public Key is sent to the SQLite database, while the Private Key is stored securely within the browser's local storage.
2. **The Challenge:** Upon passing Factor 2, the server generates a random 32-byte string (a cryptographic nonce) and stores it in the challenges table.
3. **The Signature:** The server sends this challenge to the client. The browser uses the hidden ECDSA Private Key to digitally sign the challenge and returns the signature.
4. **Verification & Deletion:** The server verifies the signature using the stored Public Key. If valid, it issues the final JWT and immediately deletes the challenge from the database, preventing replay attacks.
- 5.



5. API Endpoint Specifications

The FastAPI backend exposes several strictly typed REST endpoints to facilitate the 3FA handshake.

- **POST /auth/register**: Accepts username and password, generates the Argon2ID hash, and returns the TOTP provisioning URI.
- **POST /auth/register-device**: Accepts the ECDSA Public Key generated by the client and binds it to the user's database record.
- **POST /auth/login-f1**: Validates the password and returns the **pending_f2** JWT.
- **POST /auth/verify-f2**: Validates the TOTP code and returns the **pending_f3** JWT.
- **POST /auth/challenge**: Generates and stores the 32-byte cryptographic nonce.
- **POST /auth/verify-f3**: Verifies the ECDSA signature, deletes the nonce, and returns the fully authorized JWT.
- **GET /feed**: A protected route that strictly requires a fully authorized JWT to return the social media feed payload.
-



6. Threat Modeling & Defense Mechanisms

This architecture successfully mitigates several high-profile attack vectors:

- **Server Database Breach:** If an attacker steals the `3fa.db` file, they gain nothing actionable. Passwords are mathematically secured via Argon2ID, and the database only holds Public Keys. The ECDSA Private Keys remain entirely on the users' physical devices, upholding the Zero-Knowledge Proof.
- **Network Snooping and Replay Attacks:** Because the cryptographic challenge generated during Factor 3 is a single-use nonce that is deleted upon verification, an attacker intercepting the network traffic cannot reuse the captured signature for future logins.
- **Phishing & Credential Theft:** If an attacker successfully phishes a user's password and clones their TOTP seed, the login will still be blocked at Factor 3. The attacker's computer does not physically possess the local ECDSA Private Key required to solve the server's challenge.
-

7. Implementation Challenges & Edge Cases

While the strict inherence model provides maximum security, it introduces challenges regarding device lifecycle management. Because the Private Key is bound to a single physical browser instance, clearing local storage or switching to a new device will result in a Factor 3 failure.

In a theoretical production deployment of this architecture, a "Master Recovery Code" system would be implemented. Users would be provided a high-entropy, offline code during registration. If a device is lost, this code would authorize the generation of a new ECDSA keypair, overwriting the old Public Key in the database and restoring access without violating the Zero-Trust constraints.

8. Conclusion

This project successfully implements the strict requirements for SEC-PRJ-7F_25, utilizing OTP, Cryptotools, and JWTs to create a mathematically rigorous defense system. By transforming the user's browser into a hardware token via asymmetric cryptography, the gateway achieves true Triple Factor Authentication without compromising user experience.